ORBIT.CyberGraph-Node

Neo4j Layer — Implementation Reference

D1 · D2 · D3 · D4 | Schema v1.1.0 | 41 Tests Passing

Team 45 · IIIT-H · Neo4j 5.18.1 · Python Driver 6.1.0

1. Implementation Overview

The Neo4j graph layer is the local knowledge store at the heart of the ORBIT.CyberGraph-Node. It integrates telemetry from Wazuh (assets, vulnerabilities), privacy scan results from Microsoft Presidio (sensitivity scores, crown jewel classification), action intelligence from ORBIT Core (ActionCards, threats), and human HITL decisions (analyst approvals and rejections). All data is anonymised before entry — no raw PII is stored.

Schema version 1.1.0 was implemented against the amended conceptual specification (Neo4j_Conceptual_Doc_v1.1_Amended.docx), which resolved 12 gaps found in the original v1.0 draft.

Deliverable	Description	Status	Key Files
D2	Schema guidance: constraints, indexes, versioning	Complete — v1.1.0 applied	`execution/schema/01_apply_schema.py`
D3	Integration contracts: ActionCard, Delta, Subscription	Complete — Contract A, B, C implemented	`execution/tests/test_contracts.py`
D4	Ingestion templates: all entity types	Complete — 10 scripts, idempotent	`execution/ingestion/`
D1	Graph ontology — amendments applied	Complete — 12 gaps resolved	`Neo4j_Conceptual_Doc_v1.1_Amended.docx`

2. Graph Schema

2.1 Node Labels

Label	Canonical Key	Source	Key Properties
Host	`host_id`	Wazuh	hostname, ip, os, agent_version, last_seen, enrollment_ts
Service	`service_id*`	Wazuh	name, port, proto, first_seen (*composite: host_id::port::proto)
Application	`app_id*`	Wazuh	name, version, vendor (*composite: host_id::name::version)
Vulnerability	`cve_id`	Wazuh / CVE feeds	cvss (0–10), published, summary
DataAsset	`asset_hash`	Presidio	sensitivity_score (0–1), pii_types, crown_jewel (bool), scan_status
Threat	`threat_id`	ORBIT Core / TI feeds	title, severity, confidence, published
ActionCard	`action_id`	ORBIT Core	action_type, status (7-state), confidence, origin
Analyst	`analyst_id`	HITL UI / local directory	name, role
Subscription	`subscription_id`	Node Agent	context_summary, status, generated_ts
DeltaEvent	`delta_id`	Node Agent	sent_status, entity_count, page_token
ExecutionEvent	`exec_id`	Node Agent / Wazuh	outcome, details, ts
SchemaVersion	`name`	Schema setup script	version, updated, notes

2.2 Relationships

Relationship	From	To	Purpose
`:RUNS`	Host	Service	Links host to services it runs
`:HAS_APP`	Host / Service	Application	Tracks installed packages for CVE matching
`:HAS_VULNERABILITY`	Host / Service / App	Vulnerability	Core asset-to-CVE link
`:RESIDES_ON`	DataAsset	Host	Locates sensitive data stores on hosts
`:AFFECTS`	ActionCard	Host / DataAsset / Service	Which assets an ActionCard targets
`:RECOMMENDS`	Threat	ActionCard	Links threat intel to recommended mitigations
`:EXPLOITED_BY`	Vulnerability	Threat	Maps CVE to known threat actor / campaign
`:ASSIGNED_TO`	ActionCard	Analyst	Pending assignment pattern (UC-06)
`:APPROVED_BY`	ActionCard	Analyst	HITL approval record
`:EXECUTED`	ActionCard	ExecutionEvent	Audit log of execution attempts
`:INCLUDED_IN`	Any node	DeltaEvent	Delta export traceability
`:GENERATED`	Subscription	Subscription	Version chain for subscription history

2.3 ActionCard Status Lifecycle

The ActionCard moves through a 7-value state machine. Transitions are enforced in code via VALID_TRANSITIONS — any invalid transition raises InvalidStateTransitionError.

Status	Meaning	Trigger	Terminal?
`received`	Arrived from Core, persisted	`Core sends ActionCard`	No
`pending`	Validated, awaiting assignment	`Auto-validate on ingest`	No
`approved`	Analyst approved execution	`HITL approval via :APPROVED_BY`	No
`executing`	Active Response triggered	`begin_execution()`	No
`completed`	Successfully executed	`record_execution_result(success)`	Yes
`rejected`	Analyst rejected	`reject_action()`	Yes
`failed`	Execution failed (system fault)	`record_execution_result(failure)`	Yes

3. File Reference

File	Phase	Purpose
`execution/schema/00_verify_environment.py`	Phase 0	Verifies Neo4j 5.x server, APOC, Python driver 6.x. Exit code 1 on failure.
`execution/schema/01_apply_schema.py`	Phase 1	Applies all 10 constraints + 9 indexes + SchemaVersion 1.1.0. Idempotent.
`execution/schema/02_verify_schema.py`	Phase 1	Asserts every constraint/index exists. Prints ✓/✗ per item.
`execution/exceptions.py`	Phase 2	Defines PrivacyViolationError, InvalidStateTransitionError, OrphanEntityError, ContractValidationError.
`execution/ingestion/ingest_host.py`	Phase 2	MERGE Host on host_id. Updates last_seen on every call.
`execution/ingestion/ingest_service.py`	Phase 2	Generates composite service_id (host_id::port::proto). Guards host existence.
`execution/ingestion/ingest_application.py`	Phase 2	Composite app_id (host_id::name::version). Guards host existence.
`execution/ingestion/ingest_vulnerability.py`	Phase 2	Validates CVE format + CVSS range. Raises OrphanEntityError if host missing.
`execution/ingestion/ingest_threat.py`	Phase 2	MERGE Threat. Optional :EXPLOITED_BY link to Vulnerability.
`execution/ingestion/ingest_dataasset.py`	Phase 2	Derives crown_jewel from env threshold. Privacy guard before any Cypher. Pending-retry path.
`execution/ingestion/ingest_actioncard.py`	Phase 2	Initial status=received. Auto-validates to pending. Unresolved targets stored, never fatal.
`execution/ingestion/ingest_subscription.py`	Phase 2	Archives previous active Subscription, creates new, :GENERATED chain.
`execution/ingestion/ingest_delta_event.py`	Phase 2	Append-only DeltaEvent CREATE. INCLUDED_IN linking. Queued/sent helpers.
`execution/ingestion/batch_ingest.py`	Phase 2	Chunked batch runner with binary-search failure isolation.
`execution/lifecycle/actioncard_lifecycle.py`	Phase 3	All 7 lifecycle transition functions. VALID_TRANSITIONS enforcement.
`execution/delta/compute_delta.py`	Phase 4	Delta computation, export to JSON, DeltaEvent tracking, soft-delete detection, acknowledge.
`execution/queries/verification_queries.cypher`	Phase 5	8 Cypher queries for Neo4j Bloom/NeoDash with test plan references.
`execution/tests/conftest.py`	Phase 6	Pytest fixtures: driver, clean_graph (wipes data, preserves schema).
`execution/tests/test_schema.py`	Phase 6	5 schema tests: constraints, indexes, version, uniqueness enforcement.

`execution/tests/test_ingestion.py`	Phase 6	28 ingestion tests: CRUD, idempotency, edge cases, privacy, crown_jewel.
`execution/tests/test_lifecycle.py`	Phase 6	11 lifecycle tests: happy path, rejection, invalid transitions, relationships.
`execution/tests/test_delta.py`	Phase 6	8 delta tests: computation, export, ack, soft-delete, privacy.
`execution/tests/test_contracts.py`	Phase 6	3 contract tests: D3 field mapping, delta schema, subscription schema.
`execution/tests/SCHEMA_HEALTH_CHECKS.md`	Phase 1	5 quick Cypher checks for ongoing schema health monitoring.
`execution/tests/MANUAL_TESTING_GUIDE.md`	Phase 6	7-section guide for manual Neo4j Browser verification.

4. Key Design Decisions

4.1 MERGE-first ingestion

All ingestion functions use MERGE on the canonical key, never bare CREATE (except append-only audit nodes: ExecutionEvent, DeltaEvent). This makes every ingestion call safe to retry and idempotent — a Wazuh agent reporting the same host 50 times creates exactly one Host node.

4.2 crown_jewel is derived, never stored as input

crown_jewel is computed in Python as sensitivity_score >= CROWN_JEWEL_THRESHOLD (from .env, default 0.7) before the Cypher call. If a payload contains a crown_jewel field it is stripped and a WARNING is logged. This ensures the classification always reflects the current Presidio score and cannot be spoofed by an external caller.

4.3 Composite keys for Service and Application

service_id = host_id::port::proto and app_id = host_id::name::version are generated internally. This prevents two different hosts both running nginx from being merged into one node, which would happen if a global UNIQUE constraint were placed on a bare service name.

4.4 PendingAction is an ActionCard pattern, not a label

UC-06 Pending Action Assignment is modelled as ActionCard(status='pending') + [:ASSIGNED_TO]->(:Analyst). No :PendingAction label exists. This avoids duplicated state and keeps the lifecycle model coherent.

4.5 Privacy guard is structural, not optional

ingest_dataasset raises PrivacyViolationError and executes zero Cypher if the location_pseudonym contains raw PII patterns (@, 9+ digit sequences). The check runs before any database call, making it impossible to accidentally persist raw PII even if upstream anonymisation fails.

4.6 Soft-delete, never physical delete

Nodes are never physically removed. soft_delete() sets deleted=True, deleted_ts, and last_updated. The delta computation includes deleted nodes as operation='delete' so ORBIT Core can remove them from its view. Physical deletion would silently break delta exports.

4.7 last_updated on every write

Delta computation queries WHERE n.last_updated > datetime($last_synced_ts). Any write that omits last_updated makes that node permanently invisible to delta exports. It is treated as a required field on par with the canonical key.

5. Integration Contracts (D3 Summary)

Contract A — ActionCard (Core → Node)

Core sends ActionCards to the Node. Required fields: action_id, origin, action_type, summary, confidence, recommended_ts, affected (hosts/assets/services). Node sets status='received' on arrival, auto-validates to 'pending'. Unresolved targets stored in ac.unresolved_targets — never a fatal error.

Contract B — Delta Export (Node → Core)

Node sends incremental changes since last sync. Required top-level fields: contract_version, delta_id, delta_event_id, generated_ts, page_token, changes[]. Each change has: entity_type, entity_id, operation (create/update/delete), properties, relationships, last_updated, source. No raw PII in payload.

Contract C — Subscription (Node → Core)

Node tells Core what ActionCard types it wants. Required fields: contract_version, subscription_id, generated_ts, context_summary (crown_jewel_count, active_vulnerability_count, top_pii_types, host_count), preferred_action_types.

Scenario	Node response	Graph action	Contract
Valid ActionCard received	200 accepted	MERGE ActionCard, AFFECTS links	A
Duplicate ActionCard (same action_id)	200 accepted	Idempotent update	A
Missing required field	400 + error list	No node created	A
Core offline during delta	Cache locally	DeltaEvent.sent_status=queued	B
Core offline during subscription	Cache locally	Subscription.status=queued	C
Contract version mismatch	409 conflict	No action	A/B/C

6. Test Suite Summary

41 tests, 0 failures, 18.22s runtime on Neo4j 5.18.1 Community.

File	Tests	Coverage
`test_schema.py`	5	Constraints, indexes, SchemaVersion, uniqueness enforcement
`test_ingestion.py`	28	All entity types, idempotency, crown_jewel derivation, privacy guard, composite keys, subscription archiving
`test_lifecycle.py`	11	Full happy path, rejection, 3 invalid transition blocks, all relationship assertions
`test_delta.py`	8	Delta computation, export, acknowledge, soft-delete detection, privacy on JSON output

`test_contracts.py`	3	D3 field mapping (Contract A), delta schema (Contract B), subscription schema (Contract C)

Running the suite

```
# From project root with venv active
pytest execution/tests/ -v --tb=short

# With coverage
pytest execution/tests/ -v --cov=execution --cov-report=term-missing

# Single file
pytest execution/tests/test_ingestion.py -v
```

7. Environment & Configuration

Variable	Default	Purpose
NEO4J_URI	bolt://localhost:7687	Bolt connection for Python driver
NEO4J_USER	neo4j	Database user
NEO4J_PASSWORD	(set in .env)	Database password — never hardcode
NEO4J_DATABASE	neo4j	Target database name
CROWN_JEWEL_THRESHOLD	0.7	sensitivity_score cutoff for crown_jewel=True
DELTA_PAGE_SIZE	200	Max entities per delta page
INGEST_BATCH_SIZE	200	Rows per transaction in batch_ingest
LOG_LEVEL	INFO	Python logging level
LOG_DIR	.tmp/logs	Log file output directory
NODE_ID	orbit-node-local-dev	Node identity in subscription payloads

Docker stack

```
# Start
docker compose up -d

# Verify healthy
docker compose ps

# Reset data only (keep schema)
docker exec -it orbit-neo4j cypher-shell -u neo4j -p <pass> "MATCH (n) WHERE NOT
'SchemaVersion' IN labels(n) DETACH DELETE n"

# Full wipe (removes volumes)
docker compose down -v && docker compose up -d
```

8. Known Constraints & Pending Items

Item	Detail	Owner / Action
Community edition	Existence constraints not supported. Schema setup skips them silently.	Upgrade to Enterprise if existence constraints required
D5 — Priority scoring	Scoring formula (CVSS × sensitivity) not yet implemented. crown_jewel threshold is a blunt proxy.	Anushka + Yashav — needed before MCP prioritization (test 41)
D6 — Ops checklist	Minimal ops procedures not yet documented.	Yashav — required for Phase 4 handover
D7 — Full documentation	Final documentation package pending D5/D6 completion.	All
Wazuh integration	Ingestion scripts are ready. Wazuh API connector not yet wired.	Sharif — ingest_host/service/vulnerability accept Wazuh-shaped payloads
Presidio integration	ingest_dataasset ready. Presidio scanner not yet connected.	Anushka — privacy guard already in place
ORBIT Core mock	Contracts A/B/C implemented. Mock Core not yet wired to ingestion scripts.	Ayush — docker-compose stub needed

last_synced_ts persistence	Currently in-memory. Production needs persistent store.	Yashav — add to .env or Neo4j metadata node